A COMPLETE, VISUAL GUIDE

R-Squared & Adjusted R-Squared

How to actually measure a regression model's fit — the honest way. Plus why "Accuracy" is the wrong tool entirely, and what to use instead when predicting numbers.

📐 R² from scratch | 🛡 The overfitting loophole | ✅ Adjusted R² fix | 🎯 Accuracy vs R²

A dark-mode tutorial · Easy language, real depth

CONTENTS

What's Inside

01	The Basics: What Is R-Squared?	Intuition
02	The Math: SS_res and SS_tot, Visually	Math
03	Reading the R^2 Score, Including Negatives	Interpretation
04	The Flaw: The Illusion of Improvement	Core Concept
05	The Fix: Adjusted R-Squared	Math
06	Worked Example: The House Price Scenario	Worked Example
07	The Golden Rule: Which One to Use	Practical
08	The Bigger Picture: Classification vs Regression	Core Concept
09	Why Accuracy Fails for Regression	Deep Dive
10	Why R^2 Is the Right Tool	Deep Dive
11	Beyond R^2: Other Regression Metrics to Know	Pro-Level
12	Limitations & Traps of R^2	Pro-Level
13	Full Python Example	Code
14	One-Page Cheat Sheet	Recap

HOW TO READ THIS

Every math section starts with plain-English intuition and ends with a picture. If you just want the "gist," read the **Intuition** boxes and look at the charts — the equations are there whenever you're ready to go deeper.

SECTION 01

The Basics: What Is R-Squared?

The single most common way to grade a regression model.

When you build a regression model — one that predicts a continuous number like a price, a temperature, or a score — you need some way to answer: **"is this model actually any good?"** **R-squared (R^2)** is the standard answer.

At its core, R^2 compares your trained model against the simplest possible baseline: a "model" that always predicts the average (mean) of the target variable, no matter what the input is. R^2 then reports **what percentage of the variance in your target variable your features actually explain**, beyond just guessing the average every time.

> **INTUITION**
>
> Imagine trying to guess people's heights with zero information — your best strategy is to always guess the average height. Now imagine you're given one more clue: their age. If that clue helps you guess much more accurately than always guessing the average, your "model" (age → height) has a high R^2. If the clue barely helps at all, R^2 stays close to zero.

The idea in one sentence

R^2 asks: "How much better is my model than just guessing the average, every single time?" It reports the answer as a percentage-like score, almost always between 0 and 1.

Term	Plain-English meaning
`SS_res`	Residual Sum of Squares — the total error your *trained model* makes
`SS_tot`	Total Sum of Squares — the total error a *baseline* "*always guess the mean*" model makes
`R²`	How much of that baseline error your model manages to eliminate

SECTION 02

The Math: SS_res and SS_tot, Visually

Two error totals, one ratio.

THE R^2 FORMULA

$$R^2 = 1 - \frac{SS_{res}}{SS_{tot}}$$

Let's unpack each half of that fraction.

RESIDUAL SUM OF SQUARES — ERROR OF YOUR ACTUAL MODEL

$$SS_{res} = \sum_{i=1}^{n} (y_i - \hat{y}_i)^2$$

Here, $\hat{y}_i$ is your model's prediction for point *i*, and y_i is the true value. You square each error (so positive and negative errors don't cancel out) and sum them all up.

TOTAL SUM OF SQUARES — ERROR OF THE "ALWAYS GUESS THE MEAN" BASELINE

$$SS_{tot} = \sum_{i=1}^{n} (y_i - \bar{y})^2$$

Here, $\bar{y}$ is simply the mean of all the true target values. This measures how spread out the real data is around its own average — it's the error you'd get from the dumbest possible baseline model.

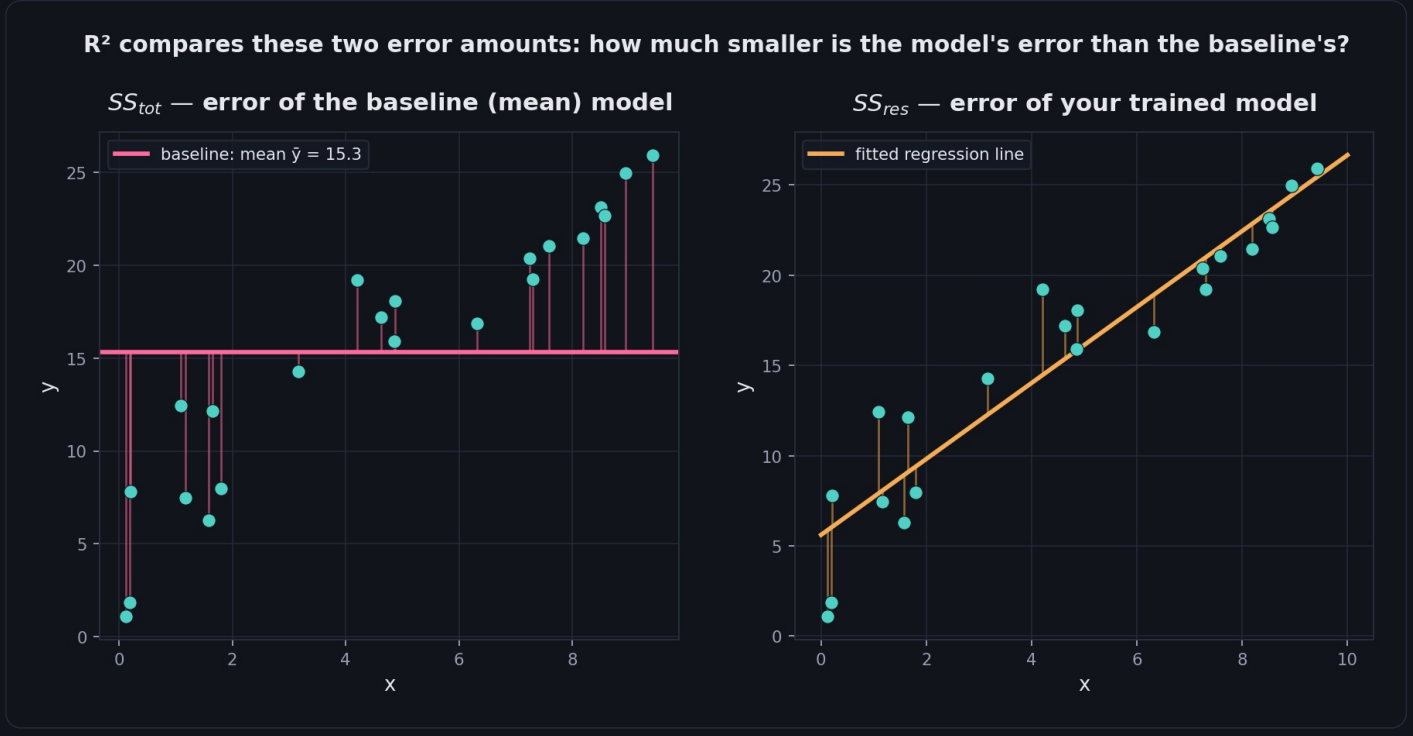


Fig 1 — Left: the pink lines show how far each point is from the flat baseline (the mean) — this total is SS_tot. Right: the orange lines show how far each point is from the fitted regression line — this total is SS_res. A good model makes the right-hand lines much shorter than the left-hand ones.

PUTTING IT TOGETHER

If your model is very good, `SS_res` will be tiny compared to `SS_tot`, so the fraction `SS_res/SS_tot` is close to 0, making `R² = 1 − (tiny number)` close to 1. If your model is no better than just guessing the mean, `SS_res ≈ SS_tot`, the fraction is close to 1, and `R² ≈ 0`.

SECTION 03

Reading the R^2 Score, Including Negatives

What the number actually tells you.

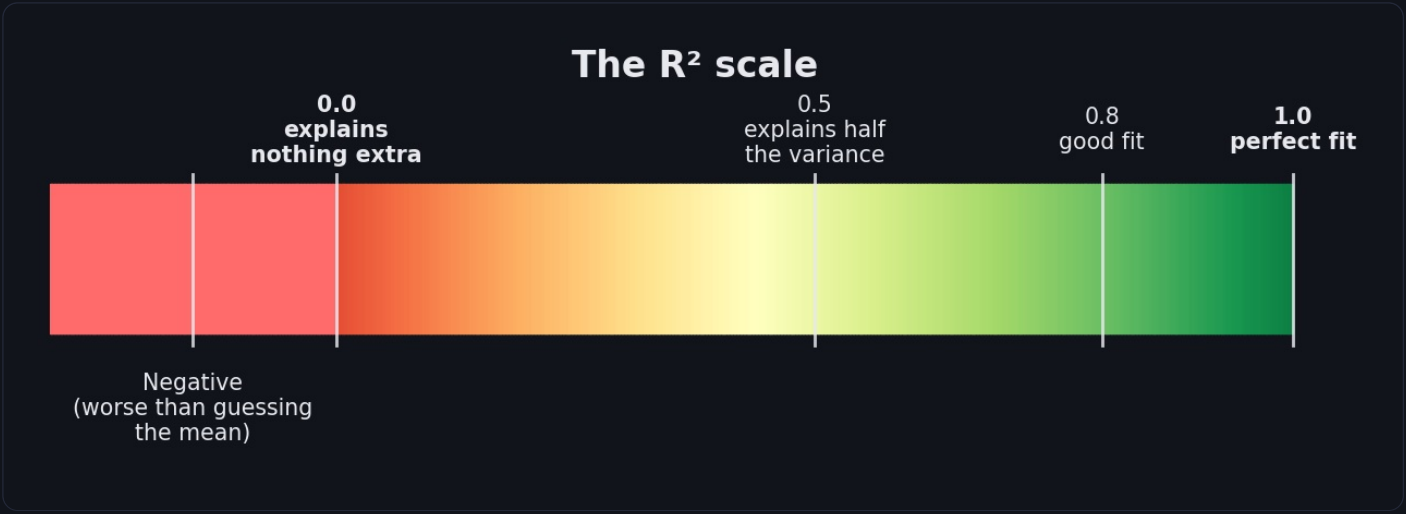


Fig 2 — The R^2 scale. Most scores you'll see in practice land between 0 and 1, but the score can technically go negative.

R^2 value	Meaning
1.0	Perfect fit — every prediction exactly matches the actual value (rare, and often a red flag for data leakage!)
0.7 – 0.99	Strong fit — your features explain most of the variance
0.3 – 0.7	Moderate fit — common in noisy real-world domains (e.g. social/behavioral data)
≈ 0	Your model explains essentially nothing beyond the average
Negative	Your model is *worse* than just guessing the mean every time — a serious warning sign, usually from a badly misspecified model or evaluating on very different data than it was trained on

A COMMON MISCONCEPTION

There's no universal "good" R^2 threshold. An R^2 of 0.3 might be genuinely impressive when predicting something as noisy as human behavior or stock returns, while an R^2 of 0.9 might be mediocre for a simple, well-understood physical process. Always judge R^2 relative to your domain, not against some fixed rule of thumb.

SECTION 04

The Flaw: The Illusion of Improvement

R²'s dirty secret: it's a one-way ratchet.

Here's the critical loophole: **standard R² mathematically can never decrease when you add more features** to a linear regression — not even if those features are complete garbage, like random noise, or your homeowner's shoe size. At absolute worst, R² stays exactly the same. Usually, it creeps up at least a little.

WHY THIS HAPPENS

Adding any new feature gives the model one more "knob" it can turn to reduce training error. Even a completely random, meaningless column can be assigned a tiny coefficient that happens to nudge a few predictions closer to their true values purely by chance. With enough useless features, this adds up — inflating R² even though the model has gained no real understanding of the data.

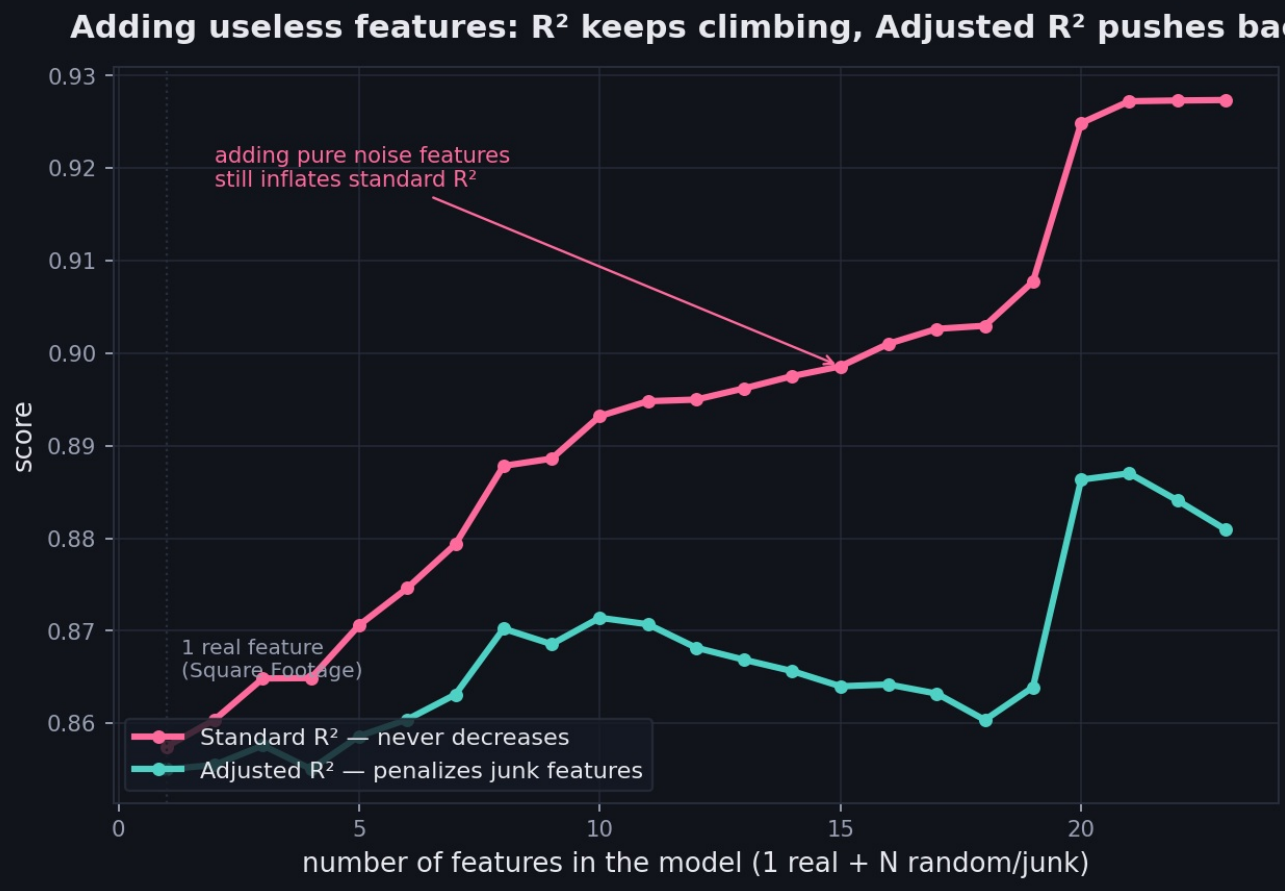


Fig 3 — Starting from one real feature, we keep adding pure random noise columns. Standard R² (pink) never goes down — it keeps climbing purely from chance. Adjusted R² (teal) recognizes the added features aren't earning their keep.

WHY THIS MATTERS

This loophole can seriously mislead you during feature selection. If you're comparing a 3-feature model against a 15-feature model using standard R² alone, the 15-feature model will very often "win" on paper — even if 12 of those features are useless — simply because R² rewards complexity for free. This is a direct path to **overfitting**: a bloated model that looks great on paper but generalizes poorly to new data.

SECTION 05

The Fix: Adjusted R-Squared

A version of R^2 that actually makes you pay for extra complexity.

Adjusted R^2 solves the loophole directly: it only goes up if a newly added feature improves the model's fit by **more than would be expected by pure random chance**. Add a genuinely useful feature, and it rewards you. Add junk, and it penalizes you.

THE ADJUSTED R^2 FORMULA

$$Adjusted\ R^2 = 1 - \left[\frac{(1 - R^2)(n - 1)}{n - k - 1}\right]$$

Symbol	Meaning
`n`	Total number of data points (samples) in your dataset
`k`	Number of independent input features (predictors) in the model
`R²`	The standard R² score, computed as usual

How the penalty mechanism works

Look closely at the denominator: `n − k − 1`. As you add more features, `k` grows, which **shrinks the denominator**. A shrinking denominator makes the fraction inside the brackets *larger* — which, after being subtracted from 1, **drags the overall Adjusted R² score down**, unless the corresponding drop in `(1 − R²)` (from a genuinely better fit) is large enough to compensate.

INTUITION

Think of Adjusted R² as R² with a "complexity tax" attached. Every new feature has to prove it's worth its cost. If it's a genuinely useful predictor, the tax is easily paid off by the improvement in fit. If it's junk, the tax exceeds the (nearly nonexistent) improvement, and your score drops.

IMPORTANT NUANCE

Adjusted R² can technically be lower than 0, and — unlike standard R² — it isn't a strict percentage/proportion anymore. It's best understood purely as a **comparison tool between models on the same dataset**, not as a standalone "percent of variance explained" figure.

SECTION 06

Worked Example: The House Price Scenario

Seeing the penalty mechanism fire in a concrete case.

SCENARIO A — ONE REAL FEATURE

You build a model to predict house prices using exactly one feature: **Square Footage**. Its R^2 comes out to **0.80** — meaning house size alone explains 80% of the variance in price. Since there's only one feature, standard R^2 is perfectly appropriate here, and Adjusted R^2 would give almost the same value.

SCENARIO B — ADD AN IRRELEVANT FEATURE

You add a second feature: the current homeowner's **shoe size** — obviously irrelevant to price. Standard R^2 creeps up to **0.81**, which looks like an improvement! But Adjusted R^2 tells a different story: it **drops to 0.78**, correctly flagging that the "improvement" wasn't real — it was just noise being fit by chance, and the complexity tax wasn't paid off.



Fig 4 — Standard R^2 is fooled by the irrelevant shoe-size feature (0.80 → 0.81). Adjusted R^2 correctly penalizes it (0.80 → 0.78), revealing that Scenario B is actually the worse model.

If you were only looking at standard R^2, you might conclude Scenario B is the better model. Adjusted R^2 protects you from that mistake — it's telling you, correctly, that Scenario A (simpler, equally predictive) is the better choice.

SECTION 07

The Golden Rule: Which One to Use

A simple rule that covers almost every situation.

Use standard R^2

...for **simple linear regression**, where you only have **one** input feature. There's no risk of the "add junk features" loophole because there's nothing extra to add.

Default to Adjusted R^2

...for **multiple regression**, where you have **two or more** input features. This is true any time you're comparing models with different numbers of features, to ensure you're rewarding genuine predictive power rather than sheer model complexity.

A quick decision guide

1. Are you comparing two or more candidate models that use a **different number of features** from each other? → Use Adjusted R^2.
2. Are you doing feature selection — trying to decide whether a new variable is actually worth including? → Use Adjusted R^2 as your guide.
3. Do you have exactly one predictor and just want a simple, interpretable "percent of variance explained" number? → Standard R^2 is perfectly fine.

IN PRACTICE

Many practitioners just report both numbers side by side. If they're close together, your features are all pulling their weight. If Adjusted R^2 is meaningfully lower than standard R^2, that gap itself is useful information — it's telling you how much of your apparent performance is inflated by feature-count alone.

SECTION 08

The Bigger Picture: Classification vs Regression

Why you can't just use "Accuracy" for everything.

A common point of confusion for beginners: **why not just use Accuracy for every model?** The answer is that Accuracy and R^2 are built for two entirely different kinds of prediction problems, and they are not interchangeable.

Classification → Accuracy

Used when predicting **distinct, discrete categories**. Examples: Is this email spam or not spam? Is this image a cat or a dog? Will this customer churn — yes or no?

Regression → R^2

Used when predicting **continuous numbers**. Examples: What will this house sell for? What will tomorrow's temperature be? How many units will we sell next month?

THE ACCURACY FORMULA

$$Accuracy = \frac{Correct\ Predictions}{Total\ Predictions}$$

Accuracy is beautifully simple: count how many predictions were exactly correct, divide by the total. This works perfectly when there are a small number of exact, discrete right answers. It falls apart completely once your target is a continuous number, as the next section shows.

SECTION 09

Why Accuracy Fails for Regression

A one-dollar mistake that Accuracy treats like a catastrophe.

Imagine a regression model predicting house prices. The true price is **$250,000**. Your model predicts **$249,999** — off by exactly one dollar. In the real world, this is an outstanding prediction.

But Accuracy only understands exact matches. Since $249,999 ≠ $250,000, Accuracy marks this prediction as **100% wrong** — treating it identically to a prediction that was off by a million dollars.

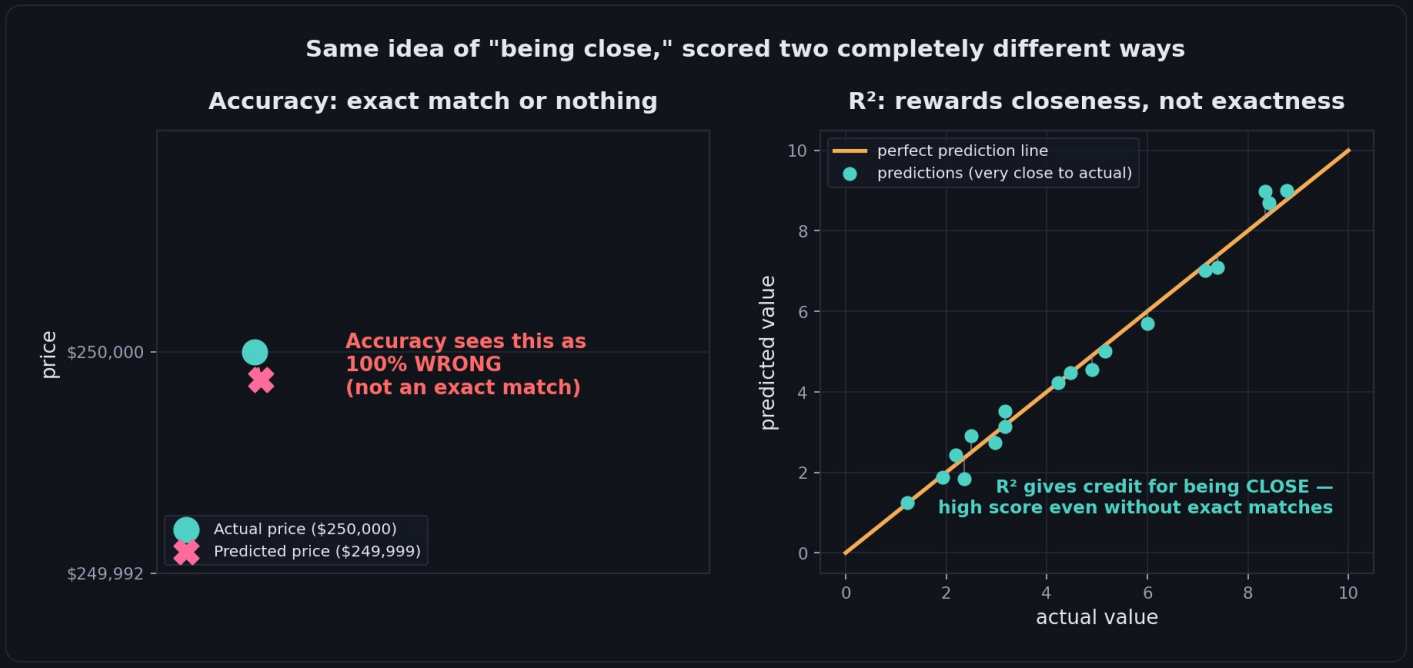


Fig 5 — Left: Accuracy can only see "exact match" or "wrong" — a $1 miss counts the same as any other miss. Right: R² measures how close predictions cluster to the true values across the whole dataset, rewarding near-perfect predictions appropriately.

THE CORE PROBLEM

Accuracy has no concept of "distance" or "closeness." It cannot distinguish between a model that's consistently very close to the truth and a model that's wildly, randomly wrong — as long as neither one hits the exact value, both score 0%. For continuous targets, that makes Accuracy essentially useless.

SECTION 10

Why R^2 Is the Right Tool

Measuring distance instead of demanding exactness.

In regression, you almost never expect to predict the exact decimal value — there are infinitely many possible values, and real-world noise guarantees some error. Instead of asking **"did we get it exactly right?"**, R^2 asks **"how close are our predictions to the real numbers, across the whole dataset?"**

Because R^2 is built from squared differences (Section 2), it naturally measures the *variance* — the spread of errors — rather than demanding perfect matches. A model that's consistently close to the truth earns a high R^2, exactly as it should.

Summary comparison

Feature	Accuracy	R-Squared (R^2)
Machine learning task	Classification	Regression
Target variable	Categories (Spam, Fraud, Yes/No)	Continuous numbers (Price, Weight, Speed)
What it evaluates	Exact matches (Right vs. Wrong)	Distance from the truth (variance and error)
Typical range	0% to 100%	Usually 0 to 1 (can go negative)
Sensitive to near-misses?	No — all misses count equally	Yes — smaller errors score much better

THE ONE-LINE TAKEAWAY

Use **Accuracy** when your answer is one of a fixed set of categories. Use **R^2** (and Adjusted R^2 for multiple features) when your answer is a number on a continuous scale. Neither tool works well outside its own domain.

SECTION 11

Beyond R^2: Other Regression Metrics to Know

R^2 is popular, but it's rarely the only metric you should look at.

R^2 tells you the *proportion* of variance explained, but it doesn't tell you the actual size of your errors in real-world units (like dollars, or degrees). For that, pair R^2 with an error metric:

MEAN ABSOLUTE ERROR (MAE) — AVERAGE ERROR SIZE, IN THE TARGET'S OWN UNITS

$$MAE = \frac{1}{n}\sum_{i=1}^{n}|y_i - \hat{y}_i|$$

MAE is easy to interpret directly: "on average, our predictions are off by $12,400" is immediately meaningful in a way R^2 alone never is.

ROOT MEAN SQUARED ERROR (RMSE) — LIKE MAE, BUT PENALIZES BIG MISSES HARDER

$$RMSE = \sqrt{\frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2}$$

Because errors are squared before averaging, RMSE punishes large individual errors more severely than MAE does — useful when big mistakes are especially costly.

Metric	Best for	Units
R^2 / Adjusted R^2	Overall "how much variance explained," comparing models	Unitless (0 to 1 scale)
MAE	Communicating typical error size simply	Same as target (e.g. dollars)
RMSE	When large errors are especially costly / outliers matter	Same as target (e.g. dollars)

PRACTICAL TIP

A strong report includes at least one variance-based metric (R^2 or Adjusted R^2) alongside one error-based metric (MAE or RMSE). Together, they answer both "how much of the pattern did we capture?" and "how far off are we, in real terms?"

SECTION 12

Limitations & Traps of R^2

Things even experienced practitioners forget.

▸ **High R^2 doesn't mean correct model.** A model can have a high R^2 while still being systematically biased — for example, missing an obvious non-linear pattern that a scatterplot of residuals would instantly reveal.

▸ **R^2 says nothing about causation.** A feature can "explain" variance statistically without having any causal relationship with the target.

▸ **Training R^2 is not the same as test R^2.** Just like accuracy, R^2 computed on training data can look great while badly overfitting — always evaluate R^2 on held-out/test data (or via cross-validation) before trusting it.

▸ **R^2 isn't well-defined for all models.** It's built around linear regression's variance decomposition; for some non-linear models, the standard SS_res/SS_tot formula can even produce unintuitive results.

▸ **A perfect R^2 of 1.0 is suspicious, not celebratory.** In real-world noisy data, a perfect score usually signals data leakage (e.g. accidentally including a feature derived from the target) rather than a flawless model.

▸ **R^2 is scale-dependent in comparisons.** You can't directly compare R^2 scores across two different datasets or two different target variables — it's only meaningful when comparing models on the *same* target and dataset.

ALWAYS PLOT YOUR RESIDUALS

Before trusting any R^2 value, plot the residuals (errors) against your predictions. Random-looking scatter around zero is healthy. Any visible curve, funnel shape, or pattern means your model is missing something that R^2 alone won't reveal.

SECTION 13

Full Python Example

Computing R^2, Adjusted R^2, and comparing feature sets.

```
# 1. Imports
import numpy as np
from sklearn.linear_model import LinearRegression
from sklearn.metrics import r2_score, mean_absolute_error, mean_squared_error
from sklearn.model_selection import train_test_split

# 2. Helper: adjusted R^2
def adjusted_r2(r2, n, k):
    return 1 - (1 - r2) * (n - 1) / (n - k - 1)

# 3. Fit Scenario A: square footage only
X_a = df[["sqft"]]
y   = df["price"]
Xa_train, Xa_test, y_train, y_test = train_test_split(X_a, y, test_size=0.2, random_state=0)
model_a = LinearRegression().fit(Xa_train, y_train)
r2_a = r2_score(y_test, model_a.predict(Xa_test))
print("Scenario A -- R^2:", r2_a,
      "| Adjusted R^2:", adjusted_r2(r2_a, len(y_test), Xa_test.shape[1]))

# 4. Fit Scenario B: square footage + an irrelevant feature
X_b = df[["sqft", "owner_shoe_size"]]
Xb_train, Xb_test, y_train, y_test = train_test_split(X_b, y, test_size=0.2, random_state=0)
model_b = LinearRegression().fit(Xb_train, y_train)
r2_b = r2_score(y_test, model_b.predict(Xb_test))
print("Scenario B -- R^2:", r2_b,
      "| Adjusted R^2:", adjusted_r2(r2_b, len(y_test), Xb_test.shape[1]))

# 5. Always pair with an error metric in real units
preds = model_a.predict(Xa_test)
print("MAE:", mean_absolute_error(y_test, preds))
print("RMSE:", np.sqrt(mean_squared_error(y_test, preds)))
```

NOTE ON STATSMODELS

If you use `statsmodels` instead of scikit-learn for linear regression, `model.summary()` reports both `R-squared` and `Adj. R-squared` automatically, alongside p-values for each coefficient — handy for quickly spotting features that aren't statistically pulling their weight.

SECTION 14

One-Page Cheat Sheet

The core idea

R^2 measures how much better your model is than just predicting the average every time. Adjusted R^2 is the same idea, but it charges a "complexity tax" for every feature you add.

Concept	Formula	Use for
R^2	$1 - SS_{res}/SS_{tot}$	Simple regression (1 feature)
Adjusted R^2	$1 - [(1-R^2)(n-1)/(n-k-1)]$	Multiple regression (2+ features)
MAE	$\text{mean}(\lvert y_i - \hat{y}_i \rvert)$	Typical error size, in real units
RMSE	$\sqrt{\text{mean}((y_i - \hat{y}_i)^2)}$	Penalizing large errors more
Accuracy	correct / total	Classification only — never regression

The golden rule

One feature → standard R^2 is fine. Two or more features → always check Adjusted R^2 too, especially when comparing models or doing feature selection.

Classification vs Regression, at a glance

Discrete categories (spam/not spam) → Accuracy. Continuous numbers (price, temperature) → R^2 / Adjusted R^2. Never mix the two up — each is meaningless on the other's type of problem.

Red flags to remember

Standard R^2 rising after adding a feature isn't proof of improvement. $R^2 = 1.0$ is suspicious, not celebratory. Always evaluate on held-out data, and always plot your residuals.

— End of tutorial —